WHERE & Filtering

Filter Exactly the Data You Need

LEARNING OUTCOME

By the end of this module you will filter rows precisely using WHERE with comparison, logical, range, pattern and NULL operators — and write complex multi-condition filters using AND, OR, NOT, IN, BETWEEN, and LIKE.

Skills You Will Gain

- Use all comparison operators: =, !=, <, >, <=, >=
- Combine conditions with AND, OR, NOT
- Filter ranges with BETWEEN and lists with IN
- Pattern match with LIKE and wildcards
- Handle NULL values correctly with IS NULL / IS NOT NULL
- Understand operator precedence and use parentheses

■ **WHERE is the most powerful SQL clause for analysts. The right filter returns exactly the data you need from billions of rows in milliseconds. Mastering WHERE is mastering SQL.**

SECTION 1

Comparison Operators

Operator	Meaning	Example
=	Equal to	WHERE status = 'active'
!= or <>	Not equal to	WHERE status != 'cancelled'
>	Greater than	WHERE price > 1000
<	Less than	WHERE age < 18
>=	Greater than or equal	WHERE score >= 90
<=	Less than or equal	WHERE quantity <= 5
BETWEEN a AND b	Inclusive range	WHERE price BETWEEN 500 AND 2000
IN (list)	Matches any in list	WHERE city IN ('Mumbai','Delhi')
NOT IN (list)	Matches none in list	WHERE status NOT IN ('cancelled','refunded')
LIKE 'pattern'	Pattern match text	WHERE name LIKE 'A%'
IS NULL	Value is NULL	WHERE phone IS NULL
IS NOT NULL	Value is not NULL	WHERE email IS NOT NULL

SECTION 2

Filtering with WHERE

```
-- Basic WHERE SELECT * FROM customers WHERE city = 'Mumbai'; SELECT * FROM orders WHERE
total_amount > 5000; SELECT * FROM products WHERE is_available = TRUE; -- AND -- BOTH conditions
must be true SELECT * FROM customers WHERE city = 'Mumbai' AND annual_spend > 10000; -- OR -- AT
LEAST ONE condition must be true SELECT * FROM customers WHERE city = 'Mumbai' OR city = 'Delhi' OR
city = 'Bangalore'; -- Cleaner with IN (equivalent to multiple OR) SELECT * FROM customers WHERE
city IN ('Mumbai', 'Delhi', 'Bangalore', 'Chennai'); -- NOT IN SELECT * FROM orders WHERE status
NOT IN ('cancelled', 'refunded', 'returned'); -- BETWEEN (inclusive on both ends) SELECT * FROM
orders WHERE total_amount BETWEEN 1000 AND 10000; SELECT * FROM orders WHERE order_date BETWEEN
'2024-01-01' AND '2024-03-31'; -- Q1 2024 -- Combined AND + OR with parentheses (ALWAYS use
parentheses with mixed AND/OR) SELECT * FROM customers WHERE (city = 'Mumbai' OR city = 'Delhi')
AND annual_spend > 50000 AND is_premium = TRUE; -- NOT SELECT * FROM customers WHERE NOT
is_premium; SELECT * FROM orders WHERE NOT (status = 'pending' OR status = 'cancelled');
```

SECTION 3

LIKE Pattern Matching

LIKE Wildcards

% (percent) -- matches ANY sequence of characters (including empty) _ (underscore)-- matches exactly ONE character Examples: LIKE 'A%' -- starts with A LIKE '%gmail%'-- contains gmail anywhere LIKE '%@gmail.com'-- ends with @gmail.com LIKE '_ohn' -- any 4-letter name ending in 'ohn' (John, Bohn...) LIKE 'A___' -- A followed by exactly 3 characters ILIKE: case-insensitive LIKE (PostgreSQL). MySQL LIKE is case-insensitive by default. Performance note: LIKE '%pattern%' cannot use indexes -- avoid for large tables.

```
-- LIKE pattern matching SELECT * FROM customers WHERE first_name LIKE 'A%'; -- starts with A
SELECT * FROM customers WHERE email LIKE '%@gmail.com'; -- gmail users SELECT * FROM products WHERE
product_name LIKE '%laptop%'; -- contains laptop SELECT * FROM customers WHERE phone LIKE
'98________'; -- starts with 98, 10 digits -- NOT LIKE SELECT * FROM customers WHERE email NOT LIKE
'%@yahoo.%'; -- NULL handling -- MUST use IS NULL, not = NULL SELECT * FROM customers WHERE phone
IS NULL; -- no phone on file SELECT * FROM customers WHERE phone IS NOT NULL; -- has phone SELECT *
FROM orders WHERE discount_code IS NULL; -- no discount applied -- COALESCE -- replace NULL with a
default value SELECT first_name, COALESCE(phone, 'No phone') AS contact_phone, COALESCE(city,
state, country, 'Unknown') AS location, COALESCE(discount_pct, 0) AS discount FROM customers; --
NULLIF -- return NULL if two values are equal SELECT NULLIF(quantity, 0) FROM inventory; -- avoid
divide-by-zero -- CASE expression -- if/else inside SQL SELECT order_id, total_amount, CASE WHEN
total_amount >= 10000 THEN 'High Value' WHEN total_amount >= 2000 THEN 'Mid Value' WHEN
total_amount >= 500 THEN 'Standard' ELSE 'Low Value' END AS order_tier, CASE status WHEN
'delivered' THEN 'Complete' WHEN 'shipped' THEN 'In Transit' WHEN 'processing' THEN 'Being
Prepared' ELSE 'Other' END AS status_label FROM orders;
```

SECTION 4

String and Date Functions in WHERE

```
-- String functions in WHERE SELECT * FROM customers WHERE UPPER(city) = 'MUMBAI'; SELECT * FROM
customers WHERE LOWER(email) LIKE '%@gmail%'; SELECT * FROM products WHERE LENGTH(product_name) >
30; SELECT * FROM customers WHERE TRIM(city) = 'Mumbai'; -- remove whitespace -- Date functions in
WHERE SELECT * FROM orders WHERE YEAR(order_date) = 2024; -- MySQL SELECT * FROM orders WHERE
EXTRACT(YEAR FROM order_date) = 2024; -- PostgreSQL SELECT * FROM orders WHERE MONTH(order_date) =
1; -- January orders SELECT * FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'; --
last 30 days SELECT * FROM orders WHERE order_date >= DATE_SUB(NOW(), INTERVAL 30 DAY); -- MySQL --
Type casting SELECT * FROM log_table WHERE CAST(log_value AS DECIMAL) > 100; SELECT * FROM events
WHERE CAST(event_date AS DATE) = '2024-01-15'; -- Operator precedence (AND binds tighter than OR)
-- This is WRONG (not what you think): SELECT * FROM orders WHERE status = 'shipped' OR status =
'delivered' AND amount > 5000; -- SQL reads it as: status='shipped' OR (status='delivered' AND
amount>5000) -- Use parentheses to be explicit: SELECT * FROM orders WHERE (status = 'shipped' OR
status = 'delivered') AND amount > 5000; -- CORRECT intent
```

■ PRACTICE Q & A

Q1. Why can't you use = NULL in SQL? Why must you use IS NULL?

A: *NULL represents unknown -- you cannot compare unknown with =. NULL = NULL is NULL (not TRUE). Use IS NULL and IS NOT NULL specifically designed to check for NULLs.*

Q2. What is the difference between IN and BETWEEN?

A: *IN matches against a specific list of values: IN ('A','B','C'). BETWEEN matches a continuous range: BETWEEN 100 AND 500 (inclusive on both ends). BETWEEN is equivalent to >= 100 AND <= 500.*

Q3. Why should LIKE '%pattern%' be avoided on large tables?

A: *Leading % prevents index usage -- the database must scan every row (full table scan). For large tables, use full-text search (FULLTEXT INDEX in MySQL, tsvector in PostgreSQL) instead.*

Q4. What does COALESCE do?

A: *Returns the first non-NULL value from its argument list. COALESCE(phone, mobile, 'No contact') returns phone if not NULL, else mobile if not NULL, else 'No contact'. Perfect for providing default values.*

Q5. What is the difference between WHERE and HAVING?

A: *WHERE filters individual rows BEFORE grouping (cannot use aggregate functions). HAVING filters groups AFTER GROUP BY (can use COUNT, SUM, AVG, etc.). WHERE runs at step 3, HAVING at step 5 of execution order.*

■ WHERE & Filtering Cheat Sheet	
`WHERE col = 'value'`	Exact match filter
`WHERE col != 'value'`	Not equal
`WHERE col BETWEEN 100 AND 500`	Inclusive range
`WHERE col IN ('A','B','C')`	Match any in list
`WHERE col NOT IN ('X','Y')`	Exclude values
`WHERE col LIKE 'A%'`	Starts with A
`WHERE col LIKE '%gmail%'`	Contains gmail
`WHERE col IS NULL`	Value is missing
`WHERE col IS NOT NULL`	Value exists
`COALESCE(col, default)`	Replace NULL with default
`CASE WHEN x THEN y END`	Conditional expression
`AND before OR`	Use () to group OR conditions